CSE 4345: Big Data Analytics

Week 7, Part 2 — Research Article & Case Study

Rokan Uddin Faruqui
Professor
Department of Computer Science & Engineering
University of Chittagong, Chittagong

CSE, PUC, Spring 2026

Part 2 Overview — Week 7

Today's Agenda

1. Paper 1 — Thusoo et al. (2009): Hive at VLDB
2. Paper 2 — Thusoo et al. (2010): Hive at Petabyte Scale
3. Case Study — Spotify Discover Weekly (2015–2018)
4. Live Exercise

Research Articles Covered

1. Thusoo, A., et al. (2009). *Hive: A Warehousing Solution over a Map-Reduce Framework*. VLDB 2009.
2. Thusoo, A., et al. (2010). *Hive: A Petabyte Scale Data Warehouse Using Hadoop*. IEEE ICDE 2010.

CLO Addressed

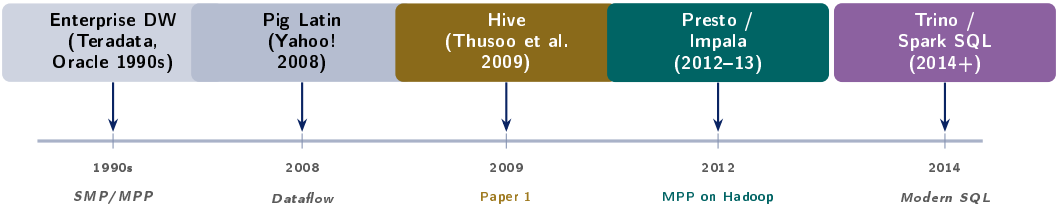
CLO3, CLO4 — Explain SQL-on-Hadoop; solve petabyte-scale analytics design problems
PLO(b,c)

Case Study

Spotify Discover Weekly (2015–2018): HiveQL + Luigi pipelines over **2 billion playlists** to generate personalised weekly mixes for **30 million** listeners.

Paper 1 — Thusoo et al. (2009): Hive at VLDB

Research Landscape: SQL-on-Hadoop Timeline



Why Hive Was Necessary

By 2008 Facebook had **hundreds of analysts** who could write SQL but not Java MapReduce. Pig Latin was more expressive than MR but still procedural. Hive translated the *declarative* SQL dialect (HiveQL) directly to MapReduce — giving every analyst self-service access to Facebook's 15 TB/day event corpus without engineering involvement.

Thusoo et al. (2009): Publication Context

Full Citation

Thusoo, A., Sarma, J. S., Jain, N., Shao, Z., Chakka, P., Anthony, S., Liu, H., Wyckoff, P., & Murthy, R. (2009). **Hive: A Warehousing Solution over a Map-Reduce Framework.** *Proceedings of the VLDB Endowment, 2*(2), 1626–1629.

Venue: VLDB 2009, Lyon, France — Rank A* (CORE)
Cited by: >3,000 (Google Scholar, 2024)
Authors: Ashish Thusoo & Joydeep Sen Sarma (Facebook Data Infrastructure team)

The Founding Problem

Facebook's Hadoop cluster held **2 PB** of data by 2009 and ran **7,500 queries/day**. Analysts wrote ad hoc

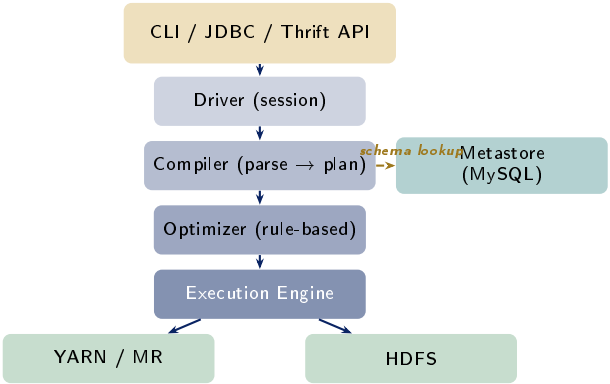
Hive's Design Goals (from the paper)

1. **Familiarity**: HiveQL must be close enough to ANSI SQL that experienced analysts need no retraining
2. **Extensibility**: custom SerDes and UDFs allow processing of any data format (logs, JSON, Thrift)
3. **Performance**: partition pruning and bucketing must reduce the data scanned for common query patterns
4. **Fault tolerance**: rely on MapReduce / HDFS for durability; Hive adds no new failure modes

Technical Contribution 1 — Hive Architecture

Four-Component Design

- **Metastore**: a relational DB (MySQL or Derby) storing table schemas, partition metadata, column statistics, and SerDe mappings. Shared across all Hive clients — the "schema-on-read" catalogue for HDFS files.
- **Driver**: receives HiveQL queries, manages the session, maintains query lifecycle
- **Compiler**: parses HiveQL → abstract syntax tree → semantic analysis (type checking, resolving column names against Metastore) → logical plan → physical plan (MapReduce DAG)
- **Execution Engine**: submits MR jobs to YARN in dependency order; collects results



Technical Contribution 2 — Metastore & SerDe

Metastore: Schema-on-Read Catalogue

The Hive Metastore stores:

- **Table definitions**: column names, types, location in HDFS, file format, compression codec
- **Partition metadata**: for a table partitioned on `(dt, country)`, the Metastore holds the HDFS path for each $(dt, country)$ value pair; enables partition pruning without listing all files
- **Bucket count**: number of buckets per partition; used to decide whether a sort-merge join avoids a shuffle
- **Column statistics**: NDV, min/max, nulls; used by cost-based optimizer (Hive 0.14+, 2015)

SerDe: Pluggable Format Layer

A **SerDe** (Serializer / Deserializer) translates raw bytes on HDFS into Hive rows and back.

SerDe	Format
LazySimpleSerDe	\t-delimited text
OpenCSVSerDe	RFC 4180 CSV
JsonSerDe	JSON (one object/line)
AvroSerDe	Avro binary + schema
ParquetHiveSerDe	Columnar Parquet
OrcSerDe	Columnar ORC
RegexSerDe	Any log via regex

UDFs: Arbitrary Extension

Paper 2 — Thusoo et al. (2010): Hive at Petabyte Scale

Thusoo et al. (2010): ICDE and Production Results

Full Citation

Thusoo, A., Sarma, J. S., Jain, N., Shao, Z., Chakka, P., Zhang, N., Antony, S., Liu, H., & Murthy, R. (2010). **Hive: A Petabyte Scale Data Warehouse Using Hadoop.** In *Proceedings of the 26th IEEE International Conference on Data Engineering (ICDE)*, pp. 996–1005. Long Beach, CA.

Venue: IEEE ICDE 2010 — Rank A (CORE)
Cited by: >1,500 (Google Scholar, 2024)
Extends: Thusoo et al. 2009 with Facebook's production Hive deployment at petabyte scale

New Contributions Over the 2009 Paper

- Detailed **query optimisation** techniques: map-side aggregation, skew join handling, and multi-group-by optimisation
- **Index support**: compact and bitmap indexes for selective query speedup
- **Column statistics** collection (`ANALYZE TABLE`) to guide query optimizer decisions
- Production benchmarks on Facebook's 2 PB cluster: 7,500+ HiveQL queries/day with <15% failure rate

Thusoo et al. (2010): ICDE and Production Results

Facebook Production Scale (2010)

Metric	Value
Data stored	2 PB
Daily growth	15 TB
Daily queries	7,500+
Active Hive users	200+ analysts
Longest query	48 hours
Most used feature	Partitioning

Key Optimization: Map-Side Aggregation

For `SELECT country, COUNT(*) FROM logs GROUP BY country`, Hive performs a **partial aggregation in the mapper** (combiner pattern) before shuffling. On a 50-million-row table with 200 countries, this reduces shuffle data by $\approx 250{,}000\times$.

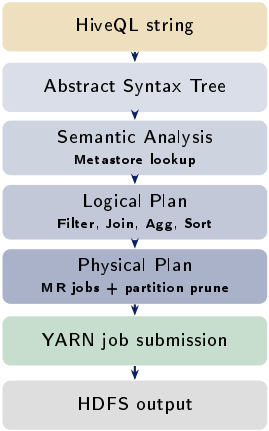
HiveQL Query Lifecycle — From SQL to MapReduce

Example: Partitioned Join Query

```
SELECT u.country, COUNT(*) AS plays FROM play_events
p JOIN users u ON p.user_id = u.user_id WHERE p.dt =
'2024-10-01' AND p.country IN ('BD','IN','US') GROUP BY
u.country ORDER BY plays DESC;
```

Compiler Steps

1. **Parse**: HiveQL → AST
2. **Semantic Analysis**: resolve `p.user_id` to `play_events.user_id : BIGINT`; check `dt` is a partition column
3. **Logical Plan**: Filter → HashJoin → GroupBy → OrderBy
4. **Partition Pruning**: only load `dt=2024-10-01/country=BD/`, `.../IN/`, `.../US/` (skip all other 365 × 200 partitions)



Impact Today — Thusoo et al. (2009, 2010)

Hive's Lasting Contributions

- **Hive Metastore** became the de facto standard catalogue for all Hadoop-ecosystem tools: Spark, Presto/Trino, Impala all read from it
- **ORC format** (2013): Hive team developed Optimized Row Columnar; 2–4× faster reads than SequenceFile; predicate pushdown within stripes
- **LLAP** (Live Long and Process, 2016): Hive-on-Tez with persistent in-memory daemon; sub-second interactive queries on multi-TB tables
- **Hive-on-Spark** (2015): execution engine swapped from MR to Spark; same HiveQL, faster execution
- **AWS Glue Catalog**: managed Metastore-compatible API used by Athena, EMR, and Redshift Spectrum
- **Apache Iceberg REST Catalog**: replaces Metastore for next-generation lakehouse architectures

Case Study — Spotify Discover Weekly (2015–2018)

Spotify's Data Challenge

Scale of the Music Listening Graph

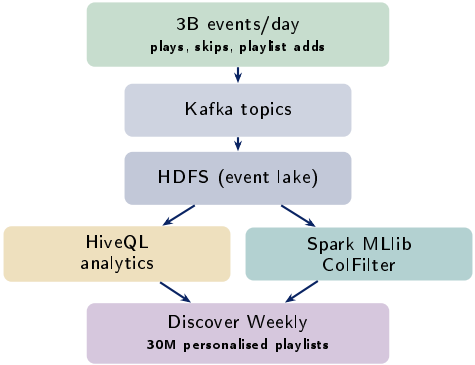
Spotify (founded 2006, 75M users by 2015) generates:

- **Streaming events**: every song play, skip, pause, repeat, and playlist add logged as a Kafka event — ≈3 billion events per day
- **30 million songs** in the catalogue; each with audio features (tempo, key, danceability, acousticness) computed by audio ML models
- **2 billion user-created playlists**: implicit feedback on what songs belong together
- All event data landed on HDFS via Kafka consumers and queried daily via HiveQL for product analytics, A/B testing, and recommendation training

Luigi: Spotify's Workflow Scheduler

Spotify open-sourced **Luigi** (Python, 2012) to orchestrate dependent HiveQL + Spark jobs as DAGs. A Luigi task declares its **output** (an HDFS path) and its **input dependencies** (other Luigi tasks). Luigi skips tasks whose output already exists — enabling incremental daily pipelines.

Spotify's Data Challenge



Discover Weekly: The Recommendation Pipeline

Three-Signal Collaborative Filtering

Discover Weekly (launched July 2015) combines three data sources, all prepared via HiveQL on Hive + Spark:

1. **Playlist co-occurrence** (implicit feedback):
   If songs A and B appear together in >N playlists, they are "similar." **HiveQL query**: `SELECT song_a, song_b, COUNT(*) FROM playlist_songs GROUP BY song_a, song_b HAVING COUNT(*) > 5`
   Over **2 billion playlists** on Hive.
2. **NLP on artist metadata** (Natural Language Processing on music blog text) — **Word2Vec** on song co-occurrence sequences, like word2vec on sentences.
3. **Audio feature similarity** — CNN-derived embeddings for 30-million songs; cosine similarity via Spotify's **Annoy** library.

Discover Weekly: The Recommendation Pipeline

Three-Signal Collaborative Filtering

Discover Weekly (launched July 2015) combines three data sources, all prepared via HiveQL on Hive + Spark:

1. **Playlist co-occurrence** (implicit feedback):
   If songs A and B appear together in >N playlists, they are "similar." **HiveQL query**: `SELECT song_a, song_b, COUNT(*) FROM playlist_songs GROUP BY song_a, song_b HAVING COUNT(*) > 5`
   Over **2 billion playlists** on Hive.
2. **NLP on artist metadata** (Natural Language Processing on music blog text) — **Word2Vec** on song co-occurrence sequences, like word2vec on sentences.
3. **Audio feature similarity** — CNN-derived embeddings for 30-million songs; cosine similarity via Spotify's **Annoy** library.

Lessons Learned & Discussion Questions

Key Lessons from Spotify's Case

1. **Implicit feedback at scale beats ratings**: Spotify's 2 billion playlists were richer signal than explicit star ratings (cf. Netflix Prize); collecting behavioural events via Kafka + HDFS is the enabling infrastructure
2. **HiveQL for feature engineering, Spark for ML**: co-occurrence aggregation (SQL) and ALS matrix factorisation (iterative) suit different engines; a single pipeline tool (Luigi) bridges them
3. **Schema design determines query speed**: partitioning `play_events` by `week` reduced the co-occurrence HiveQL from 8 hours to 2 hours
4. **Cold-start is still hard**: new songs with no playlist appearances get audio-feature

Discussion Questions (CLO3, CLO4)

1. Spotify's co-occurrence query runs `GROUP BY song_a, song_b` over 2 billion playlist rows. Estimate the output size if there are 30M songs. Propose a **partitioning + bucketing** scheme for the `playlist_songs` Hive table that would reduce the shuffle size for this query. *[CLO4 — Apply]*
2. The ALS collaborative filtering model is a matrix factorisation over a 75M-user $\times$ 30M-song matrix. At 4 bytes per float and rank $k = 50$, calculate the model size and explain whether it fits in a 5,000-node YARN cluster's aggregate RAM. *[CLO4 — Analyse]*

Live Exercise

Live Exercise — HiveQL on Local Hadoop (or SparkSQL)

Task (25 minutes, pairs)

Simulate a Hive session using **PySpark's SQL interface** (which reads the same Hive Metastore).

Dataset: a 10,000-row synthetic `play_events` CSV (generate with the skeleton below).

Goals:

1. Create a partitioned Spark table (partition on `dt`).
2. Run the co-occurrence query and time it.
3. Add a **broadcast join** hint for the songs dimension table and compare timings.
4. Use `EXPLAIN` to print the physical plan and identify where partition pruning occurs.

PySpark SQL Skeleton

```
from pyspark.sql import SparkSession
import random, datetime

spark = SparkSession.builder \
        .appName("HiveDemo") \
        .getOrCreate()
spark.setLogLevel("WARN")

# Generate synthetic data
rows = []
for i in range(10_000):
    rows.append({
      "user_id":    random.randint(1, 500),
      "song_id":    random.randint(1, 200),
      "playlist_id":random.randint(1, 1000),
      "dt":         "2024-10-01"
    })

df = spark.createDataFrame(rows)
df.createOrReplaceTempView("play_events")

# Co-occurrence query
result = spark.sql("""
```

References

Research Articles

- Thusoo, A., et al. (2009). Hive: A warehousing solution over a map-reduce framework. *Proc. VLDB Endowment, 2*(2), 1626–1629.
- Thusoo, A., et al. (2010). Hive: A petabyte scale data warehouse using Hadoop. *Proc. IEEE ICDE*, 996–1005.

Textbooks [TW], [SA], [TE]

- **[TW]** White, T. (2015). *Hadoop: The Definitive Guide*, 4th ed. (Ch. 17 — Hive). O'Reilly.
- **[SA]** Acharya, S., & Chellappan, S. (2015). *Big Data and Analytics* (Ch. 7 — Data Warehousing on Hadoop). Wiley.
- **[TE]** Erl, T., Khattak, W., & Buhler, P. (2016). *Big Data Fundamentals* (Ch. 9 — SQL-on-Hadoop). Prentice Hall.

Case Study Sources

- Spotify Engineering. (2015). *How Spotify Discover Weekly Works*. labs.spotify.com/2015/08/06

Closing — Week 7, Part 2

Key Takeaways

- Thusoo et al. (2009) gave the world a **declarative interface over distributed storage** — HiveQL's impact was democratisation: 200 analysts replaced 20 Java engineers
- The **Metastore** is Hive's most durable contribution — it outlived MR-based Hive execution and is now the universal catalogue for Spark, Presto, Trino, Athena, and Delta Lake
- Spotify's case demonstrates that **SQL (HiveQL) and iterative ML (Spark ALS)** are complementary in a recommendation pipeline — neither alone is sufficient at billion-playlist scale
- **Schema design is a first-class skill**: the right partition column (week vs. song vs. user) changes query latency by hours on petabyte data

Quote of the Week

"Hive provides a simple and optimizable query language, familiar to all data warehouse users, while retaining the scalability of Hadoop."

— Thusoo et al. (2009)

Part 1 Preview — Week 8

Kafka, Sqoop, Flume, Lambda/Kappa Architecture

- Kafka topics, partitions, consumer groups
- Exactly-once semantics
- Lambda: batch + speed + serving layers